

Tracking Files Lifecycle

- Create a file (app.txt)
- Check repository status
- Stage file using git add
- Commit changes using git commit
- Modify file and view differences using git diff
- Commit updated changes

Create dev branch, Switch branches, Make changes in dev, Switch back to main, Compare branches.

->

```
mkdir karan
```

```
cd karan
```

```
git init
```

```
echo "Hello World" > app.txt
```

```
git status
```

```
git add app.txt
```

```
git status
```

```
git commit -m "Added app.txt"
```

```
echo "New line added" >> app.txt
```

```
git diff
```

```
git add app.txt
```

```
git commit -m "Updated app.txt"
```

Branching and Isolation

- Create a dev branch using git branch
 - Switch branches using git checkout
 - Make changes in dev branch
 - Switch back to main branch
 - Compare branches
- Outcome: Student understands branch isolation and parallel development.

->

```
git branch dev
```

```
git checkout dev
```

```
echo "hello here new in dev branch " > app.txt
```

```
git checkout main
```

```
cat app.txt
```

Merging and Conflict Handling

- Merge dev branch into main using git merge
- Perform fast-forward merge
- Create conflicting changes in two branches
- Resolve merge conflicts manually
- Commit resolved changes

->

STEP 1: Create project

```
mkdir mergedemo
```

```
cd mergedemo
```

```
git init
```

STEP 2: Create file and first commit

```
echo "Start" > app.txt
```

```
git add .
```

```
git commit -m "Initial commit"
```

STEP 3: Create dev branch

```
git branch dev
```

STEP 4: Switch to dev and make change (for fast-forward)

```
git checkout dev
```

```
echo "Change from dev" >> app.txt
```

```
git add .
```

```
git commit -m "Dev change"
```

STEP 5: Fast-forward merge (no conflict)

```
git checkout main
```

```
git merge dev
```

STEP 6: Make change in main (to create conflict later)

```
echo "Main branch change" >> app.txt
```

```
git add .
```

```
git commit -m "Main change"
```

STEP 7: Switch to dev and make different change

git checkout dev

echo "Conflicting change from dev" >> app.txt

git add .

git commit -m "Dev conflicting change"

STEP 8: Try merging (this will cause conflict)

git checkout main

git merge dev

STEP 9: Open file to see conflict

cat app.txt

STEP 10: Manually edit file (remove <<<<<< ===== >>>>>> and keep correct lines)

nano app.txt

Example final content inside file:

Start

Change from dev

Main branch change

Conflicting change from dev

STEP 11: Mark conflict resolved

git add app.txt

git commit -m "Resolved merge conflict"

STEP 12: Verify history

```
git log --oneline
```

Working with Remote Repositories

- Understand local vs remote repositories
- Configure remote using git remote add
- Clone repository using git clone
- Fetch updates using git fetch
- Pull changes using git pull
- Push changes using git push

Outcome: Student can synchronize local and remote repositories.

->

Add remote repository (connect local repo to GitHub)

```
git remote add origin https://github.com/username/repo.git
```

Check remote connection

```
git remote -v
```

Push local code to remote repository

```
git push -u origin main
```

Clone a remote repository (create local copy)

```
git clone https://github.com/username/repo.git
```

Fetch updates from remote (download only, no merge)

git fetch

Pull changes from remote (download + merge)

git pull origin main

Push new changes to remote

git push origin main

GitHub Collaboration Workflow

- Create repository on GitHub
- Push local code to remote
- Clone repository to another directory
- Create a feature branch and push it
- Create pull request and review changes
- Merge pull request

->

STEP 1: Connect local repo to GitHub (repo already created on GitHub)

git remote add origin <https://github.com/karansinh-solanki/practise.git>

STEP 2: Push local code to remote

git push -u origin main

STEP 3: Clone same repo in another directory (simulate another user)

cd ..

```
git clone https://github.com/karansinh-solanki/practise.git  
cd practise
```

STEP 4: Create feature branch

```
git checkout -b feature-branch
```

STEP 5: Make changes in feature branch

```
echo "Feature work" >> app.txt
```

```
git add .
```

```
git commit -m "Added feature"
```

STEP 6: Push feature branch to GitHub

```
git push origin feature-branch
```

STEP 7: (Manual on GitHub)

Go to GitHub → Create Pull Request → Review changes → Merge

STEP 8: After merge, update local main branch

```
git checkout main
```

```
git pull origin main
```

Undoing Changes & Recovery

- Undo unstaged changes using `git checkout`
- Unstage files using `git reset`
- Amend commits using `git commit --amend`
- Use `git reset` (soft, mixed, hard)

- Use git revert for safe undo
- Recover commits using git reflogOutcome: Student can safely undo and recover changes.

->

UNDO UNSTAGED CHANGES (discard changes in file, back to last commit)

git checkout -- app.txt

UNSTAGE FILE (remove from staging, keep changes in file)

git reset app.txt

AMEND LAST COMMIT (change message or add new changes)

git commit --amend -m "Updated commit message"

RESET TYPES

SOFT RESET (move HEAD, keep changes staged)

git reset --soft HEAD~1

MIXED RESET (default: unstage changes, keep in working directory)

git reset HEAD~1

HARD RESET (delete commit + changes permanently)

git reset --hard HEAD~1

SAFE UNDO USING REVERT (creates new commit to undo)

git revert HEAD

SEE HISTORY OF ALL MOVES (very important for recovery)

git reflog

RECOVER LOST COMMIT USING REFLOG

git reset --hard HEAD@{1}

· Merge dev branch into main, perform fast-forward merge, create conflicting changes, Resolve conflicts, Commit resolved changes.

· Create feature branch, Make incremental commits, Push branch to remote, Update branch with latest develop changes, Merge back to develop

->

=====

PART 1: MERGE + CONFLICT

=====

Create dev branch

git branch dev

Switch to dev and make change (for fast-forward)

git checkout dev

echo "Dev change 1" >> app.txt

git add .

git commit -m "Dev change 1"

Fast-forward merge (no conflict)

git checkout main

git merge dev

Create conflict: change in main

echo "Main change" >> app.txt

git add .

git commit -m "Main change"

Change in dev (same file → conflict)

git checkout dev

echo "Dev conflicting change" >> app.txt

git add .

git commit -m "Dev conflicting change"

Merge → conflict will occur

git checkout main

git merge dev

Fix conflict manually (edit file)

Remove <<<<<< ===== >>>>>> and keep correct lines

nano app.txt

Mark resolved and commit

git add app.txt

git commit -m "Resolved merge conflict"

```
# =====
```

```
# PART 2: FEATURE WORKFLOW
```

```
# =====
```

```
# Create feature branch
```

```
git checkout -b feature-branch
```

```
# Make incremental commits
```

```
echo "Feature line 1" >> app.txt
```

```
git add .
```

```
git commit -m "Feature step 1"
```

```
echo "Feature line 2" >> app.txt
```

```
git add .
```

```
git commit -m "Feature step 2"
```

```
# Push feature branch to remote
```

```
git push origin feature-branch
```

```
# Update branch with latest develop (or main)
```

```
git checkout main
```

```
git pull origin main
```

```
git checkout feature-branch
```

```
git merge main
```

```
# Merge feature back to main (or develop)
```

```
git checkout main
```

```
git merge feature-branch
```

```
# Push final merged code
```

```
git push origin main
```

Git Hooks for Automation

- Understand client-side hooks
- Create pre-commit hook to run lint checks
- Create pre-push hook to run tests
- Automate validation before commits
- Explore post-merge hook usage

->

```
# Go to hooks folder
```

```
cd .git/hooks
```

```
# =====
```

```
# PRE-COMMIT HOOK (runs before commit)
```

```
# =====
```

```
echo '#!/bin/sh'
echo "Pre-commit running" > pre-commit
chmod +x pre-commit
```

```
# =====
# PRE-PUSH HOOK (runs before push)
# =====
```

```
echo '#!/bin/sh'
echo "Pre-push running" > pre-push
chmod +x pre-push
```

```
# =====
# POST-MERGE HOOK (runs after merge)
# =====
echo '#!/bin/sh'
echo "Post-merge executed" > post-merge
chmod +x post-merge
```

to check go to out cd ..

```
echo "test" >> app.txt
git add .
git commit -m "check hook"
```

output :

Pre-commit running

git push origin main

Pre-push running

git merge dev

Post-merge executed